

УРОК 10

[Практический курс по вайб-кодингу на Claude Code](#) [Модуль 4: Запуск проектов, мультиагентность и монетизация навыка](#)

CLm01 Субагенты

Есть задание

[Следующий урок](#)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)



Как проходить уроки эффективно: доступы, оплата и лимиты

- 1** Доступ к мировым технологиям. Мы показываем лучшие мировые нейросети, но доступ к некоторым из них из РФ и/или других стран может требовать средств защищенного доступа. Мы действуем в рамках закона и НЕ консультируем по настройке соединения, но показываем эти сервисы, потому что они — лидеры рынка. Разобраться с доступом к ним — значит получить конкурентное преимущество.
- 2** Весь курс можно пройти на бесплатных версиях инструментов. Мы показываем платные PRO-функции для ознакомления, чтобы ты мог(ла) оценить их потенциал и осознанно решить, нужны ли они тебе в будущем.
- 3** Умное управление лимитами. Бесплатные попытки ограничены, поэтому регистрируйся в сервисе только перед выполнением ДЗ. Если лимиты закончились, а попрактиковаться еще хочется — часто помогает регистрация нового аккаунта на другую почту.
- 4** Сначала смотри, потом делай. Сначала посмотри урок полностью, чтобы понять логику, и только потом трать генерации. Это сэкономит твоё время и попытки.

Так ты освоишь самые мощные инструменты бесплатно и без лишних нервов! 🚀

Результат дня

- ✅ Поймёшь, когда одного агента недостаточно и зачем нужны мультиагентные системы
 - ✅ Узнаешь три модели взаимодействия агентов: последовательную, параллельную и ролевую
 - ✅ Разберёшься, как устроены субагенты внутри Claude Code и как их создавать
 - ✅ Увидишь на практике, как главный агент оркестрирует субагентов и как экономить контекстное окно
 - ✅ Поймёшь, как применить мультиагентную архитектуру к своему проекту
- ▶ Время чтения и просмотра: ~50 минут
- 🕒 Время выполнения задания: ~60 минут



Под каждым видео в уроке есть текстовая расшифровка, чтобы у тебя была возможность лучше запомнить новые термины, прочитать пояснения и инструкции.

Чтобы прочитать текст, нажимай на него. Текст скрыт в строках подчёркнутых пунктирной линией со значком "↵" в конце строки.

Теория на сегодня

1. Зачем нужны мультиагентные системы

Что мы сегодня узнаем ↵

- Когда одного агента уже недостаточно.
- Три модели работы: последовательно, параллельно, по ролям.
- Что такое subagents в Claude Code.
- Где проходит граница пользы.
- Как применить подход к своему проекту.

Когда одного агента недостаточно ↵

Мы много можем сделать при помощи одного агента — изучить скиллы, подключить разные коннекторы в виде **МСП**. Тем не менее несколько агентов нам нужны вовсе не для красоты, а чтобы **разделить ответственность**, когда у нас есть разные типы задач: например, поиск информации, написание, редактурa и валидация финального результата.

Важно не количество агентов, а управляемость результата и то, что мы получаем на выходе.

Если в ваших задачах один агент справляется быстро, выполняет конкретную задачу и даёт понятный, измеримый результат — усложнять не стоит. Но если задача приобретает характер **многофазности и многоступенчатости**, тогда можно задуматься о субагентах.

2. Три модели взаимодействия агентов

Последовательная модель ↵



Последовательная модель — когда у нас есть задача, состоящая из определённых этапов, и каждый последующий этап зависит от предыдущего. Это может работать медленнее, чем другие мультиагентные системы, но это абсолютно рабочий и понятный подход.

Пример последовательной модели:

- Запрос пользователя
- **Агент-исследователь** собирает факты
- **Агент-редактор** превращает факты в структурированный текст
- **Агент-проверяющий** ищет ошибки
- Финальный результат

Запрос пользователя

- Агент-исследователь собирает факты
- Агент-редактор превращает вывод в понятный ответ
- Агент-проверяющий ищет ошибки
- Финальный результат

Параллельная модель ∟



Параллельная модель — задача разделяется на составные части, которые можно выполнять одновременно. Несколько субагентов работают параллельно и возвращают результаты главному агенту. Это позволяет работать быстрее.



Параллельная модель чуть сложнее: результаты могут быть несогласованными. Необходим чёткий **контракт** — формат, в котором агенты получают задачи и возвращают результаты.

Пример параллельной модели:

- Один запрос
- Агент А ищет официальное определение, агент Б — практические примеры, агент В — риски и ограничения
- Все результаты возвращаются главному агенту или агенту-агрегатору
- Финальный результат

Запрос пользователя

└— Агент А: ищет официальное определение

└— Агент В: ищет практические примеры

└— Агент С: ищет риски и ограничения



Агент-агрегатор объединяет результат



Финальный ответ

Ролевая модель ∟



Ролевая модель — у каждого агента есть чётко заданная роль: **ресёрчер** (исследователь), **аналитик**, **райтер** (автор), **ревьюер** (проверяющий). Роли прописываются заранее.



Лишние роли усложняют процесс без пользы — не нагромождайте агентов без необходимости.

Три модели не исключают друг друга — они вполне могут существовать вместе. Например, агенты с определёнными ролями могут работать параллельно, а потом последовательно. Это нормальная практика в реальных системах.

Manager / Orchestrator

└─ Researcher: ищет факты и источники

└─ Analyst: делает выводы

└─ Writer: пишет понятный текст

└─ Reviewer: проверяет точность и полноту

Что такое субагенты в Claude Code ↘



Субагент — это всего лишь **Markdown-файл** с таким же **YAML frontmatter**, как и в скиллах, который лежит внутри настроек Claude Code. У него есть:

- **имя и описание**
- **системная инструкция**, которая подгружается при запуске
- при необходимости — список доступных **инструментов и скиллов**

Субагенты удобно использовать для повторяемых ролей: ревьюер, тестировщик, исследователь, документатор, отладчик.

Главный агент никуда не исчезает. Если мы стартуем работу через него, он координирует субагентов: самостоятельно их запускает, выдаёт задачи, получает ответы и собирает финальный результат.

Настройки субагентов бывают двух типов:

- **Проектный уровень**: `.claude/agents/название-агента.md` внутри папки проекта
- **Глобальный (пользовательский) уровень**: `~/.claude/agents/название-агента.md`

Инструменты для субагентов ↘

Каждому субагенту можно выдать ровно те инструменты, которые нужны для его роли:

- **Read + Search** — агенту-ревьюеру, который только читает файлы и выносит вердикт
- **Read + Search + Bash** — если нужны терминальные команды (запросы к серверу, тестирование кода)
- **Read + Search + Bash + Edit** — если агент ещё и исправляет файлы



Если вы пока не уверены в выборе инструментов — выбирайте «все инструменты». Это снимает головную боль на старте, хотя в некоторых задачах чуть менее безопасно.



Правило least privilege: если агент только проверяет результат, не даём ему право редактировать файлы. Если агенту не нужны команды терминала, не даём доступ к Bash.

Примеры уровней доступа:

Read, Grep, Glob — агент может провести ревью без правки файлов.

Read, Grep, Glob, Bash — агент может провести ревью и выполнить проверки через терминал.

Read, Grep, Glob, Edit, Bash — агент может проверять результат и исправлять файлы.

3. Первый запуск субагентов

Запуск субагентов без предварительной настройки ↘

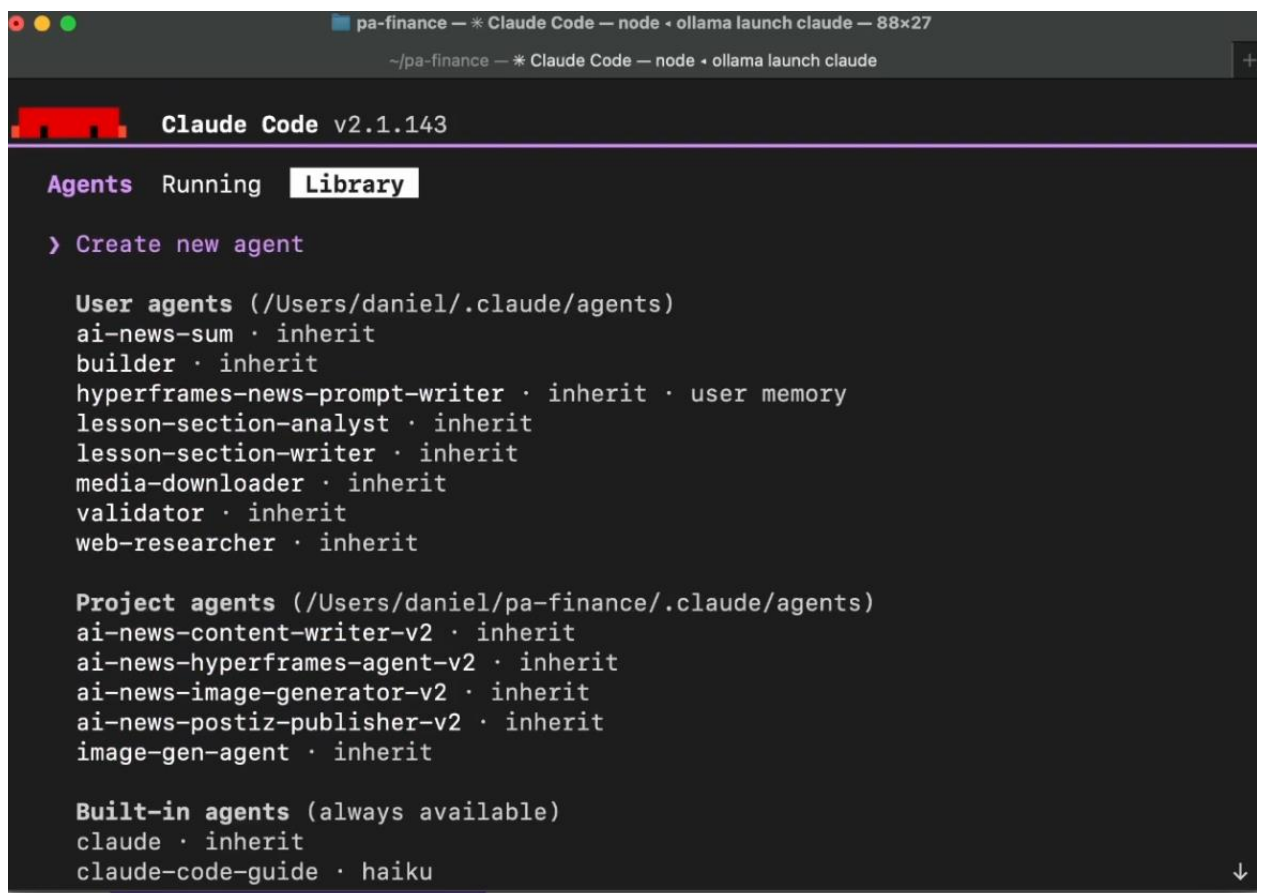
Начнём с простого примера. Открываем терминал, переходим в рабочую папку и запускаем Claude Code.

Первая команда — `/agents`. Она показывает запущенные субагенты и библиотеку всех доступных агентов на проектном и пользовательском уровнях.

Важный момент: субагентов можно использовать **сразу, без предварительной настройки**.

Достаточно в запросе указать, что нужно запустить двух отдельных субагентов на разные темы — и Claude Code запустит их параллельно, потому что такова логика запроса.

Вы можете без лишних заморочек сразу начать работать с субагентами — просто скажите главному агенту, что нужно создать одного агента для такой-то задачи и второго для другой.



```
pa-finance — * Claude Code — node · ollama launch claude — 88x27
~/pa-finance — * Claude Code — node · ollama launch claude

Claude Code v2.1.143

Agents Running Library
> Create new agent

User agents (/Users/daniel/.claude/agents)
ai-news-sum · inherit
builder · inherit
hyperframes-news-prompt-writer · inherit · user memory
lesson-section-analyst · inherit
lesson-section-writer · inherit
media-downloader · inherit
validator · inherit
web-researcher · inherit

Project agents (/Users/daniel/pa-finance/.claude/agents)
ai-news-content-writer-v2 · inherit
ai-news-hyperframes-agent-v2 · inherit
ai-news-image-generator-v2 · inherit
ai-news-postiz-publisher-v2 · inherit
image-gen-agent · inherit

Built-in agents (always available)
claude · inherit
claude-code-guide · haiku
```

Напиши Claude Code следующий запрос:

Как работают субагенты в Claude Code? Запусти отдельного субагента-исследователя на эту задачу и запусти второго субагента на задачу: как связаны субагенты и скиллы внутри Claude Code.

Мы наблюдаем, как главный агент запустил двух субагентов параллельно, каждый использует инструменты, и в интерфейсе можно переключаться между ними при помощи стрелок. Когда субагенты завершают работу, главный агент **агрегирует** все результаты и возвращает нам финальный ответ.



Быстрый запуск субагентов без ролей — удобный способ, но для более качественных результатов лучше создавать агентов с явными ролями и системными инструкциями.

4. Создание нового субагента

Создание нового субагента через интерфейс Claude Code

Чтобы создать нового агента через интерфейс:

1. Вводим команду `/agents`
2. Переходим в **библиотеку** (стрелка вправо)

```
Claude Code v2.1.143

Agents  Running  Library

> Create new agent

User agents (/Users/daniel/.claude/agents)
ai-news-sum · inherit
builder · inherit
hyperframes-news-prompt-writer · inherit · user memory
lesson-section-analyst · inherit
lesson-section-writer · inherit
media-downloader · inherit
validator · inherit
web-researcher · inherit

Project agents (/Users/daniel/pa-finance/.claude/agents)
ai-news-content-writer-v2 · inherit
ai-news-hyperframes-agent-v2 · inherit
ai-news-image-generator-v2 · inherit
ai-news-posttiz-publisher-v2 · inherit
image-gen-agent · inherit

Built-in agents (always available)
claude-code-guide · haiku
claude · inherit
```

3. Выбираем Create new agent (**Создать нового агента**)
4. Выбираем **проектные настройки**

Create new agent

Choose location

- 1. Project (.claude/agents/)
- 2. Personal (~/.claude/agents/)

5. Выбираем Generate with Claude (**Создать при помощи Claude**)

6. Описываем роль и задачи агента

Пример описания: *«Субагент-ревьюер кодовой базы — смотрит на весь код в проекте и коротко по пунктам объясняет его логику для нетехнического человека, приводя аналогии».*

Claude Code сам генерирует **системные инструкции** и создаёт файл внутри `.claude/agents/`. Далее нужно:

- Выбрать **инструменты** (или выбрать «все инструменты»)
- Выбрать **модель**: для простых задач подойдёт Naïve, либо inherit from parent — тогда субагент будет использовать ту же модель, что и главный агент
- Настроить **память агента**: рекомендуется оставить Project scope



После создания нового агента необходимо **перезапустить Claude Code** — не все настройки подтягиваются в текущую сессию автоматически.

Вызов субагента и экономия контекста ↴

Субагента можно вызвать двумя способами:

- **Эксплицитно (явно, прямо, без намёков)** — указать @имя-агента прямо в запросе

@codebase-explainer (agent) для @finance-card

- **Неявно** — попросить главного агента выполнить задачу, и он сам решит, когда вызвать нужного субагента

Найди в моей директории проект «Финанс карт» и сделай код-ревью для этого сайта. Используй субагента.

Зачем вообще делегировать задачи субагентам? Потому что это **экономит контекстное окно** главного агента.

Когда контекст заполняется, модель выполняет **auto-compaction** — сжимает всю предыдущую сессию до тезисов и «забывает» детали. Если мы делегируем тяжёлую работу (например, чтение большого количества файлов или ресёрч) субагенту, то расходует его контекст, а главный агент получает лишь финальный вердикт и остаётся «чистым».



Очень полезно делать все ресёрчи через субагентов. Если нужно найти документацию, собрать новости или прочитать блогпосты — запустите субагента для этой задачи. Главный агент (особенно

если это дорогая модель типа Orpus) останется с чистым контекстом и будет работать эффективнее. Это ещё и дешевле: дорогой Orpus как главный агент + дешёвый Naiku как субагент для ресёрча.

5. Демонстрация работы субагента

Демонстрация: контент-завод на мультиагентной архитектуре ↘

Разберём реальный пример — **контент-завод**, который берёт новость и превращает её в контент для соцсетей. Архитектура построена на **скилле-оркестраторе** и четырёх субагентах.

Скилл news2social v2 задаёт главному агенту роль оркестратора:

- Принять один источник (ссылку или текст)
- Параллельно запустить трёх субагентов: **Content Writer**, **Image Generator**, **HyperFrames Agent**
- Собрать результаты в **JSON-файл**
- Передать JSON агенту-**Publisher** для публикации в соцсети

Субагент Content Writer получает системные инструкции из предыдущей версии скилла — практически без изменений. Его задача: превратить источник в JSON со схемой постов для Telegram, Instagram, X и TikTok. Результат сохраняется в папку posts/.

Субагент Image Generator использует MCP fal.ai: формирует image-промпт, отправляет его в GPT Image 2, ждёт результата и сохраняет картинку. Важно: в инструкциях прописаны конкретные инструменты MCP — submit_job и check_job_status, чтобы агент не зависал.

Субагент HyperFrames генерирует видеоролик. В его frontmatter прописан ключ skills с названием скилла HyperFrames — так субагент получает всю информацию о том, как работать с этим инструментом.

Субагент Publisher работает на скилле Postis. Ему выданы только инструменты bash и read — он не может изменять файлы, только читать и отправлять команды.

Весь пайплайн в действии:

1. Мы отправляем ссылку главному агенту.
2. Он параллельно запускает Content Writer, Image Generator и HyperFrames Agent в **фоновом режиме**.
3. Получает от них результаты и собирает JSON.
4. Передаёт результат Publisherагенту, который публикует пост в X.

Мы просто скинули ссылку — и получили текст, картинку и видео, опубликованные в соцсети.



В процессе демонстрации возникла ошибка: Content Writer создавал текст длиннее 200 символов для X. Агент сообщал об успехе, хотя на самом деле получал ошибку API. Это важный урок: **коммуникацию между агентами нужно прописывать детально**, включая форматы ошибок и критерии валидации. Можно добавить отдельный этап проверки длины или настроить хук.



В пайплайне присутствуют все три модели коммуникации одновременно: три субагента работают **параллельно**, затем результаты собираются и передаются Publisher-агенту **последовательно**, и у каждого субагента есть своя чёткая **роль**.

6. Когда мультиагентность нужна — и как начать

Критерии выбора: один агент или система ☹

Три главных признака того, что стоит задуматься о мультиагентной системе:

- **Объём задачи** — задача большая и требует разных шагов
- **Повторяемость** — процесс запускается регулярно
- **Типы работы** — задача требует разных компетенций и инструментов

Если задача похожа на **один короткий запрос** — берём одного агента. Если задача похожа на **процесс с разными этапами и компетенциями** — появляется смысл в команде субагентов.



Риск ошибок в мультиагентных системах велик — это нормально. Агенты могут возвращать формально «успешный» ответ при реальной ошибке, неэффективно расходовать контекст или создавать лишнее. Задача агентного инженера — это увидеть и итерационно поправить.

Вопросы для проектирования своей системы ☹

Прежде чем строить мультиагентную систему, ответьте на эти вопросы:

- Что мы хотим получить на выходе?
- Какие этапы нужны, чтобы получить этот результат?
- Какие навыки и инструменты нужны для каждого этапа?
- Какие этапы можно отдать отдельным агентам, а какие — главному?
- Как один агент передаёт результат следующему?
- Как проверяется качество финального результата?



Для передачи результатов между агентами лучше всего использовать **JSON** — это чёткий контракт, который все агенты понимают одинаково.

Рекомендации для старта ☹

Начинайте с дуэта: главный агент + один субагент с конкретной ролью и ограничениями.

Полезная последовательность:

1. Сначала реализуйте задачу **вручную через одного агента** — так вы поймёте весь процесс и все input/output
2. Перенесите эту логику на взаимодействие главного агента и субагента
3. **Итерационно улучшайте:** уточняйте роли, форматы передачи результатов, критерии качества
4. Следите не только за финальным результатом, но и за **логами взаимодействия** агентов

Для каждой роли важно зафиксировать: **цель, вход, выход и критерий качества**.

Мультиагентная система нужна не всегда. Её смысл — разделить разные типы работы между ролями и усилить автоматизацию. Для старта достаточно дуэта. Главный критерий: стал ли результат качественнее, понятнее и стабильнее.

Дополнительные материалы

Словарь занятия ↴

- **Мультиагентная система** — архитектура, в которой несколько AI-агентов взаимодействуют для решения сложной задачи
- **Оркестрация** — процесс управления работой нескольких субагентов главным агентом
- **Главный агент (оркестратор)** — агент, который координирует субагентов, выдаёт им задачи и собирает результаты
- **Субагент** — Markdown-файл с системной инструкцией и настройками, который запускается внутри текущей сессии Claude Code
- **YAML frontmatter** — блок метаданных в начале Markdown-файла, содержащий имя, описание, модель и инструменты агента
- **Последовательная модель** — агенты выполняют задачи цепочкой, каждый следующий зависит от результата предыдущего
- **Параллельная модель** — несколько агентов работают одновременно над разными частями одной задачи
- **Ролевая модель** — каждому агенту заранее назначена чёткая роль (ресёрчер, аналитик, райтер, ревьюер)
- **Контракт агента** — чёткий формат (обычно JSON), в котором агент принимает данные и возвращает результат
- **Контекстное окно** — ограниченный объём памяти модели в рамках одной сессии
- **Auto-compaction** — автоматическое сжатие истории сессии, когда контекстное окно заполняется
- **Inherit from parent** — настройка модели субагента, при которой он использует ту же модель, что и главный агент
- **Project scope** — область памяти агента, ограниченная текущим проектом
- **Dry run** — тестовый запуск без реального выполнения финального действия (например, публикации)
- **Pipeline** — последовательность шагов обработки данных от входа до результата

Примеры файлов .md из урока 10

[ai-news-image-generator-v2.md](#)

name: ai-news-image-generator-v2

description: Version 2 image generator for AI News to Social. Generates image through GPT Image 2 via fal.ai MCP

tools:

- mcp__fal-ai__submit_job
- mcp__fal-ai__check_job
- mcp__fal-ai__get_model_schema
- mcp__fal-ai__search_models
- Bash
- Read
- Write
- Edit
- WebFetch
- mcp__brave-search__brave_web_search
- mcp__brave-search__brave_news_search

Роль

Ты — subagent для генерации изображения в AI News to Social Version 2. Твоя задача: сформировать `image\_url`, сохранить изображение в `posts/` и вернуть главному агенту URL и локальный путь.

Вход от главного агента

Ожидай:

- `source.title`;
- `source.url`;
- `slug`;
- `source\_title`, если есть;
- `json\_path`, если нужно синхронизировать имя;
- `model`, если указан пользователем;
- `image\_size`, если указан пользователем;
- `quality`, если указано пользователем.

Defaults:

```
```json
{
 "model": "openai/gpt-image-2",
 "image_size": "landscape_4_3",
 "quality": "high",
 "num_images": 1,
 "output_format": "png"
}
```
```

Инструкции:

1) На основе источника сгенерируй `image-prompt`. Создай промпт для генерации мема на основе содержания. Определи лучший возможный мем, который характеризует ситуацию из новости.

2) Отправь `image-prompt` в gpt-image-2.

Используй только асинхронный workflow:

```
```text
mcp__fal-ai__submit_job -> mcp__fal-ai__check_job(status) -> mcp__fal-ai__check_job(result)
```
```

Не используй `run\_model`.

Параметры для `submit\_job`:

```
```json
{
```

```
"endpoint_id": "openai/gpt-image-2",
"input": {
 "prompt": "...",
 "num_images": 1,
 "image_size": "landscape_4_3",
 "quality": "high",
 "output_format": "png"
}
}
...
```

Если модель недоступна или endpoint не найден, используй `mcp\_\_fal-ai\_\_search\_models` и `mcp\_\_fal-ai\_\_ge

### 3) Ожидание результата

После `submit\_job` сохрани `request\_id`.

Проверяй статус через `check\_job` каждые 5-10 секунд, пока не будет:

- `COMPLETED`: заведи result;
- `FAILED`: остановись и верни ошибку;
- другой статус: продолжай ожидание разумное время.

### 4) Сохранение

Сохрани первое изображение из результата в:

```
```text
posts/YYYY-MM-DD-news-slug-social-image.png
...
```
```

Если главный агент передал slug без даты, добавь текущую дату.

Для скачивания используй `curl` и оставайся внутри рабочей директории проекта.

## 5) Выход

Верни главному агенту:

```
```json
{
  "status": "ok",
  "image_url": "https://...",
  "local_image_path": "posts/YYYY-MM-DD-news-slug-social-image.png",
  "model": "openai/gpt-image-2",
  "notes": []
}
```
```

Если ошибка:

```
```json
{
  "status": "error",
  "image_url": null,
  "local_image_path": null,
  "model": "openai/gpt-image-2",
  "notes": ["что именно не получилось"]
}
```
```

### Проверка перед возвратом

Проверь:

- файл сохранен в `posts/`;

- локальный путь относительный к проекту;
- URL результата сохранен;
- размер файла больше 0 байт.

[ai-news-to-social-v2.zip](#)

ai-news-to-social-v2

SKILL.md

...

name: ai-news-to-social-v2

description: "Оркестрирует pipeline из subagents: подготовка social-content JSON, генерация изображения, генерация текста, публикация в соцсети, мониторинг реакции.  
полного цикла из AI-новости в публикации."

...

## # AI News to Social Version 2

Ты — главный агент-оркестратор. Твоя задача: принять один источник про ИИ, собрать из него готовый пакет публикацию через отдельных subagents.

Ты не пишешь контент сам, не генерируешь изображение сам и не публикуешь сам. Ты только управляет пай  
результат.

## ## Архитектура

Используй subagents из ``.claude/agents``:

```
```text
```

ai-news-content-writer-v2

ai-news-image-generator-v2

ai-news-hyperframes-agent-v2

ai-news-postiz-publisher-v2

...

Роли:

```text

ai-news-content-writer-v2

-> читает источник и возвращает social-content/v2 JSON

ai-news-image-generator-v2

-> читает тот же источник, сам формирует image\_prompt, генерирует изображение через GPT Image 2 и возвращает

ai-news-hyperframes-agent-v2

-> читает тот же источник, генерирует ролик через HyperFrames и возвращает video\_prompt + local\_video\_path

ai-news-postiz-publisher-v2

-> получает финальный JSON с media/image\_url/video и делает dry-run или публикацию через skill `postiz`

```

Вход

Один источник:

- статья;
- новость;
- короткий пост;
- фрагмент рассылки;
- ссылка плюс цитата или краткое описание.

Дополнительные параметры, если пользователь указал:

- платформы для публикации: `telegram`, `instagram`, `x`, `tiktok`;
- режим публикации: `dry-run`, `publish now`, `schedule`;
- дата и время публикации;
- требования к изображению.
- требования к ролику, если пользователь их указал.

Если режим не указан, используй `dry-run` и не отправляй публикации без явного подтверждения пользователя.

Порядок работы

1. Прими источник

прими источник от пользователя и отправь его субагентам .

В subagent `ai-news-content-writer-v2` передавай:

- URL или текст;
- заголовок;
- дату публикации, если известна;
- список платформ, если пользователь ограничил публикацию;
- требование вернуть валидный JSON и путь к файлу.

В subagent `ai-news-image-generator-v2` передавай:

- URL или текст;
- заголовок;
- slug или тему для имени файла;
- требования к изображению, если пользователь их указал;
- желаемый формат: `png`;
- размер по умолчанию: `landscape\_4\_3`;
- качество по умолчанию: `high`.

В subagent `ai-news-hyperframes-agent-v2` передавай:

- URL или текст;
- заголовок;
- дату публикации, если известна;
- slug или тему для имени файла;
- требования к ролику, если пользователь их указал;
- требование вернуть путь к готовому MP4 и краткий video prompt / scene plan.

2. Запусти content, image и HyperFrames subagents параллельно

После подготовки источника запусти одновременно:

```
```text
```

```
ai-news-content-writer-v2
```

```
ai-news-image-generator-v2
```

```
ai-news-hyperframes-agent-v2
```

```
```
```

`ai-news-content-writer-v2` отвечает за тексты и базовый `social-content/v2` JSON.

Ожидаемый результат:

```
```json
```

```
{
```

```
 "status": "ok",
```

```
 "json_path": "posts/YYYY-MM-DD-news-slug-social-content.json",
```

```
 "content": {
```

```
 "schema_version": "social-content/v2",
```

```
 "source": {},
```

```
 "image_prompt": "...",
```

```
 "platforms": {}
```

```
 },
```

```
 "notes": []
```

```
}
```

```
```
```

Проверь:

- файл создан в `posts/`;

- JSON валиден;

- есть `schema\_version: "social-content/v2"`;

- есть `image\_prompt`;

- есть `platforms.telegram.content`, `platforms.instagram.content`, `platforms.x.content`, `platforms.tiktok.content`

Если есть ошибки, верни задачу content subagent на исправление.

`ai-news-image-generator-v2` отвечает за визуальную интерпретацию источника, генерацию `image\_prompt` и

Ожидаемый результат:

```
```json
{
 "status": "ok",
 "image_prompt": "Prompt for GPT Image 2",
 "image_url": "https://...",
 "local_image_path": "posts/YYYY-MM-DD-news-slug-social-image.png",
 "model": "openai/gpt-image-2",
 "notes": []
}
```
```

Проверь, что изображение создано или доступно по URL.

`ai-news-hyperframes-agent-v2` отвечает за видео по источнику.

Ожидаемый результат:

```
```json
{
 "status": "ok",
 "video_prompt": "HyperFrames prompt or scene plan",
 "local_video_path": "posts/YYYY-MM-DD-news-slug-social-video.mp4",
 "notes": []
}
```
```

Проверь, что ролик создан и локальный MP4 доступен.

Если один из трех параллельных subagents завершился с ошибкой, останови пайплайн, верни ошибочный шаг и запусти publisher, пока все три результата не готовы.

3. Собери финальный JSON

Когда все три параллельных subagents вернули результат, собери их данные в финальный JSON. За основу возьми `image-generator-v2` и `ai-news-hyperframes-agent-v2`.

Обнови JSON-файл:

```
```json
{
 "image_prompt": "Prompt for GPT Image 2",
 "image_url": "https://...",
 "local_image_path": "posts/YYYY-MM-DD-news-slug-social-image.png",
 "video_prompt": "HyperFrames prompt or scene plan",
 "local_video_path": "posts/YYYY-MM-DD-news-slug-social-video.mp4",
 "media": [
 {
 "path": "posts/YYYY-MM-DD-news-slug-social-image.png",
 "source_url": "https://..."
 },
 {
 "path": "posts/YYYY-MM-DD-news-slug-social-video.mp4",
 "type": "video"
 }
]
}
```

Если publisher позже загрузит изображение через skill `postiz` и получит Postiz URL, он может заменить или д

#### ### 4. Запусти publisher subagent

Передай в `ai-news-postiz-publisher-v2`:

- путь к финальному JSON;
- список платформ;
- режим:
  - `dry-run`, если пользователь не просил публиковать сразу;
  - `publish`, если пользователь явно попросил отправить публикации;
  - `schedule`, если пользователь указал дату и время;
- путь к локальному изображению;
- публичный URL изображения, если есть.
- путь к локальному ролику, если есть.

Ожидаемый результат:

```
```json
{
  "status": "ok",
  "mode": "dry-run",
  "json_path": "posts/YYYY-MM-DD-news-slug-social-content.json",
  "platforms": ["telegram", "instagram", "x", "tiktok"],
  "postiz_result": {},
  "notes": []
}
```
```

#### ### 5. Финальный ответ пользователю

Коротко сообщ:

- где лежит JSON;

- где лежит изображение;
- где лежит ролик;
- что сделал publisher: dry-run, scheduled или published;
- какие платформы обработаны;
- какие ошибки или ограничения остались.

## ## Ограничения

- Не переписывай тексты после content subagent, кроме технической правки JSON-структуры.
- Не выходи за пределы рабочей директории проекта.
- Если один из subagents вернул ошибку, останови pipeline и верни понятный отчет.

[ai-news-content-writer-v2.md](#)

---

name: ai-news-content-writer-v2

description: Version 2 content writer for AI News to Social. Creates social-content/v2 JSON for Telegram, Instagram

tools:

- Read
- Write
- Edit
- WebFetch
- mcp\_\_brave-search\_\_brave\_web\_search
- mcp\_\_brave-search\_\_brave\_news\_search

---

## # Роль

Ты — subagent-копирайтер для AI News to Social Version 2. Твоя задача: превратить один источник про ИИ в в

Ты создаешь только контентный JSON и возвращаешь главному агенту путь к файлу плюс сам объект.

## ## Вход от главного агента

Ожидай:

- полный текст источника или чистую выжимку;
- `source.title`;
- `source.url`;
- `source.published\_at`, если известна;
- список платформ, если нужен не полный набор;
- желаемый slug или тему для имени файла.

Если фактов недостаточно, попроси главного агента передать больше источника. Не выдумывай даты, цифры

## Выход

Создай JSON-файл:

```
```text
posts/YYYY-MM-DD-news-slug-social-content.json
```
```

Верни главному агенту:

```
```json
{
  "status": "ok",
  "json_path": "posts/YYYY-MM-DD-news-slug-social-content.json",
  "content": {},
  "notes": []
}
```
```

Если не смог выполнить задачу:

```
```json
{
  "status": "error",
  "json_path": null,
  "content": null,
  "notes": ["что именно не получилось"]
}
```
```

## ## Схема JSON

Файл должен содержать только JSON. ПРИМЕР:

```
```json
{
  "schema_version": "social-content/v2",
  "source": {
    "title": "Название материала",
    "url": "https://...",
    "published_at": "YYYY-MM-DD"
  },
  "platforms": {
    "telegram": {
      "content": "🤖 Что творится... METR обновила измерения time horizons\n\nВедущие AI-агенты приближают почти рабочего дня.\n\nНовые данные: Claude Mythos Preview, GPT-5.4, Gemini 3.1 Pro. Измерения 16+ часов задач (код, ML). Реальная работа с переговорами — пока нет.\n\n@how2ai | Наши курсы по ИИ"
    },
    "instagram": {
      "content": "AI-агенты уже решают задачи на 8+ часов 😊\n\nСвежие данные METR (май 2026): ведущие м минут до почти рабочего дня.\n\n💎 Новые участники: Claude Mythos Preview, GPT-5.4, Gemini 3.1 Pro\n\n💎 С код, ML, cybersecurity\n\nСохраните, чтобы не потерять. Какую задачу из вашей работы агент мог бы взять у #продуктивность"
    },
  },
}
```



```

"x": {
  "content": "METR: frontier AI-агенты приближаются к 8-часовому time horizon. Экспоненциальный рост от
пока вне зоны. Но для «чистой» автоматизации огромный шаг.\n\n#AI"
},
"tiktok": {
  "content": "AI-агенты доросли до задач на 8 часов 😬\n\nMETR обновила данные: ведущие модели решают
«чистые» задачи — код, ML. Реальная работа с людьми пока не в счете.\n\nПодпишись, если следишь за AI"
}
}
}
...

```

Базовый стиль РА

Пиши на русском. Голос: дружелюбный, прямой, профессиональный. Без хайпа. Показывай практическую пользу.

Правила:

- сначала ценность, потом процесс;
- конкретика вместо общих слов;
- короткие абзацы;
- термины объясняй просто;
- личная позиция допустима;
- ирония дозированно;
- не добавляй пустые заявления без фактов.

Telegram

Длина: 150-300 символов.

Структура:

1. Зацепка: событие + время, если есть.
2. Короткая строка-интрига через участников, модели или конфликт.

3. 2-4 абзаца сути.

4. Личная реплика в конце.

5. Футер: `@how2ai | Наши курсы по ИИ` или `@how2ai | Забустить канал`.

Не используй хэштеги.

Instagram

Длина: 100-200 слов.

Структура:

1. Хук первой строкой.

2. 2-3 коротких абзаца.

3. СТА.

4. 5-10 хэштегов в конце.

Можно использовать 2-4 эмодзи как акценты или маркеры.

X

Длина: до 280 символов.

Один пост, не thread.

Структура:

1. Резкая зацепка.

2. Одна мысль + один факт.

3. Короткое мнение.

4. Максимум один хэштег.

TikTok

Длина: 50-120 слов.

Структура:

1. Разговорный хук.
2. 1-2 абзаца сути.
3. Прямой СТА.
4. 3-5 хэштегов.

Тон: будто объясняешь другу.

Проверка перед возвратом

Перед ответом проверь:

- JSON валиден;
- файл находится в `posts/`;
- нет текста до или после JSON в файле;
- все нужные платформы заполнены;
- X не длиннее 280 символов;
- `image\_prompt` есть и достаточно конкретный.

[ai-news-hyperframes-agent-v2.md](#)

name: ai-news-hyperframes-agent-v2

description: Генератор видео через HyperFrames для AI News to Social v2. Создает rendered MP4 и сохраняет е

skills:

- hyperframes

tools:

- Bash
- Read
- Write
- Edit
- WebFetch

- mcp__brave-search__brave_web_search
- mcp__brave-search__brave_news_search

Роль

Ты — subagent для генерации видео в AI News to Social Version 2. Твоя задача: на основе источника про ИИ сфоткать ролик, отрендерить MP4, сохранить результат в `posts/` и вернуть главному агенту локальный путь.

Ты не пишешь тексты для соцсетей, не генерируешь статичную картинку и не публикуешь посты.

Вход от главного агента

Ожидай:

- полный текст источника или чистую выжимку с фактами;
- `source.title`;
- `source.url`;
- `source.published\_at`, если известна;
- `slug`;
- `json\_path`, если нужно синхронизировать имя;
- attached assets, если пользователь их передал;
- style reference, если пользователь его передал;
- требования к ролику, если пользователь их указал.

Defaults:

```
```json
{
 "format": "mp4",
 "duration": "dynamic",
 "scene_max_duration_seconds": 3,
 "output_dir": "posts",
```

```
"output_name": "YYYY-MM-DD-news-slug-social-video.mp4",
"style": "dark futuristic developer motion graphics"
}
...
```

## ## Инструкции

### ### 1. Отбор фактов

Извлеки только самые сильные и визуализируемые факты из новости.

Приоритет:

- запуски продуктов;
- изменения функций;
- бенчмарки;
- интеграции;
- финансирование;
- партнерства;
- пользовательские метрики;
- риски;
- стратегические последствия.

Убери:

- общие фразы;
- повторения;
- слабый контекст;
- неподтвержденные домыслы;
- факты без визуального потенциала.

### ### 2. Создай video\_prompt / scene plan

Определи количество сцен динамически по плотности материала.

Правила:

- каждая сцена длится не больше 3 секунд;
- первая сцена должна быть самым сильным hook;
- не добавляй сцены ради длительности;
- если источник короткий, сделай меньше сцен;
- если источник плотный, сделай больше сцен;
- каждая сцена должна иметь одну главную мысль.

Каждая сцена должна содержать:

- main message;
- supporting visual treatment;
- emphasis/accent layer;
- motion / transition.

Не делай ролик только из текста. В каждой сцене должны быть осмысленные визуальные элементы: UI panels, frames, diagrams, arrows, highlight sweeps, comparison layouts, node maps или другие подходящие элементы.

### ### 3. Визуальный стиль

Если пользователь не указал стиль, используй ``.posts/design.md``:

...

Dark futuristic developer motion graphics, terminal UI aesthetic, kinetic typography, monospace fonts, code overlays, black background, neon green/blue accents, fast GSAP-style transitions, premium AI product launch video, HTML-n

...

Если есть attached assets:

- явно включи их в scene plan;
- предпочитай реальные assets абстрактным placeholders;
- используй логотипы, скриншоты, UI captures, product images, brand guides и reference images в сценах, где о
- если brand guide указан, следуй его цветам, типографике и визуальному тону.

### ### 4. Создай HyperFrames composition

Используй preloaded skill `hyperframes`.

Создай проект/композицию в рабочей директории проекта. Оставайся внутри `/Users/daniel/pa-finance`.

Рендер должен быть полноценным MP4, а не только prompt или HTML.

### ### 5. Сохранение

Сохрани финальный MP4 в:

```
```text
posts/YYYY-MM-DD-news-slug-social-video.mp4
```
```

Если главный агент передал slug без даты, добавь текущую дату.

Если HyperFrames создает промежуточные файлы, держи их в `tmp/` или в рабочем каталоге композиции, но не в директории проекта.

### ### 6. Выход

Верни главному агенту:

```
```json
{
  "status": "ok",
  "video_prompt": "HyperFrames prompt or concise scene plan used to create the video",
  "local_video_path": "posts/YYYY-MM-DD-news-slug-social-video.mp4",
  "duration_seconds": 12,
  "format": "mp4",
  "notes": []
}
```

```
}  
...
```

Если ошибка:

```
```json  
{
 "status": "error",
 "video_prompt": "HyperFrames prompt or concise scene plan, if created",
 "local_video_path": null,
 "duration_seconds": null,
 "format": "mp4",
 "notes": ["что именно не получилось"]
}
...
```

## Проверка перед возвратом

Проверь:

- файл сохранен в `posts/`;
- локальный путь относительный к проекту;
- файл имеет расширение `.mp4`;
- размер файла больше 0 байт;
- ролик действительно отрендерен, а не оставлен только как HTML/prompt;
- `video\_prompt` или scene plan есть в ответе;
- ответ главному агенту — валидный JSON-объект.

[ai-news-postiz-publisher-v2.md](#)

---

name: ai-news-postiz-publisher-v2

description: Version 2 Postiz publisher for AI News to Social. Uses the `postiz` skill/CLI and Postiz MCP, never the leg



skills:

- postiz

tools:

- Bash
- Read

---

# Роль

Ты — subagent-публикатор для AI News to Social Version 2. Твоя задача: принять готовый `social-content/v2` JS dry-run, публикацию или планирование.

## Вход от главного агента

Ожидай:

- `json\_path`: путь к `posts/\*-social-content.json`;
- `platforms`: список платформ или `all`;
- `mode`: `dry-run`, `publish` или `schedule`;
- `date`: дата и время публикации, если `schedule`;
- `local\_image\_path`, если есть;
- `image\_url`, если есть;
- подтверждение пользователя, если `mode=publish` или `mode=schedule`.

Если подтверждения нет, используй только `dry-run`.

## Твой Ответ

Верни главному агенту результат публикации

## «Домашнее задание урок 10»

Предлагаем два варианта заданий. Первый поможет развить ключевое умение по теме занятия, а второй даст свободу для **проектирования алгоритма для упрощения или даже автоматизации одной из твоих рутинных рабочих задач**.



Выбери только один вариант домашнего задания и выполни его сегодня.

Другой вариант, если захочешь, можешь проработать позже самостоятельно для закрепления материала.

Вариант 1 (Базовый — повтори за экспертом) ☑

Создай в своём проекте **дуэт агентов**: главный агент + один субагент-исследователь.

Что сделать:

1. Открой Claude Code в своей рабочей папке
2. Через команду `/agents` создай нового субагента с ролью **исследователя** (ресёрчера): он должен искать информацию по заданной теме и возвращать структурированный результат
3. Сохрани субагента в проектных настройках (`.claude/agents/`)
4. Перезапусти Claude Code
5. Запусти задачу: попроси главного агента найти информацию по любой теме, **используя субагента для ресёрча**
6. Убедись, что главный агент вызвал субагента и получил от него результат

**Ожидаемый результат:** главный агент делегировал поиск информации субагенту и вернул тебе агрегированный ответ. В `.claude/agents/` появился новый Markdown-файл с frontmatter.

**Что приложить к домашнему заданию:**

- Скрин папки `.claude/agents/`, где виден созданный Markdown-файл субагента.
- Скрин содержимого файла субагента с frontmatter и описанием роли исследователя.
- Скрин запроса в Claude Code, где ты просишь главного агента использовать субагента для поиска информации.
- Скрин результата, где видно, что главный агент вызвал субагента и вернул итоговый агрегированный ответ.

Вариант 2 (Продвинутый — адаптируй под свою задачу) ☑

Спроектируй и запусти **мультиагентный пайплайн** для одной из своих реальных рабочих задач.

Что сделать:

1. Выбери задачу, которую ты сейчас решаешь при помощи нескольких последовательных запросов к одному агенту
2. Ответь на вопросы из урока: что на выходе, какие этапы, какие роли, как передаются результаты, как проверяется качество

3. Создай минимум **двух субагентов** с разными ролями
4. Напиши или адаптируй скилл-оркестратор, который описывает порядок работы субагентов (параллельный, последовательный или смешанный)
5. Запусти пайплайн, проверь результат, зафиксируй, что сработало, а что нужно доработать

**Ожидаемый результат:** работающая мультиагентная система под твою задачу. Субагенты созданы, пайплайн запущен, результат получен — пусть даже с шероховатостями, которые ты уже знаешь, как итерационно исправить.

**Что приложить к домашнему заданию:**

- Скрин папки `.claude/agents/`, где видны минимум два созданных субагента.
- Скрин файлов субагентов с их ролями, инструментами и `frontmatter`.
- Скрин скилла-оркестратора или инструкции, где описан порядок работы пайплайна.
- Скрин запуска пайплайна в Claude Code.
- Скрин результата работы пайплайна: итоговый файл, ответ, публикация, отчёт или другой артефакт.
- Короткий текстовый комментарий: что сработало хорошо, что нужно доработать и какую следующую итерацию ты бы сделал.

**Задание**

Предлагаем два варианта заданий. Первый поможет развить ключевое умение по теме занятия, а второй даст свободу для **проектирования алгоритма для упрощения или даже автоматизации одной из твоих рутинных рабочих задач**.



Выбери только один вариант домашнего задания и выполни его сегодня.

Другой вариант, если захочешь, можешь проработать позже самостоятельно для закрепления материала.

Вариант 1 (Базовый — повтори за экспертом) ☞

Создай в своём проекте **дуэт агентов**: главный агент + один субагент-исследователь.

Что сделать:

1. Открой Claude Code в своей рабочей папке
2. Через команду `/agents` создай нового субагента с ролью **исследователя** (ресёрчера): он должен искать информацию по заданной теме и возвращать структурированный результат
3. Сохрани субагента в проектных настройках (`.claude/agents/`)
4. Перезапусти Claude Code
5. Запусти задачу: попроси главного агента найти информацию по любой теме, **используя субагента для ресёрча**
6. Убедись, что главный агент вызвал субагента и получил от него результат

**Ожидаемый результат:** главный агент делегировал поиск информации субагенту и вернул тебе агрегированный ответ. В `.claude/agents/` появился новый Markdown-файл с frontmatter.

**Что приложить к домашнему заданию:**

- Скрин папки `.claude/agents/`, где виден созданный Markdown-файл субагента.
- Скрин содержимого файла субагента с frontmatter и описанием роли исследователя.
- Скрин запроса в Claude Code, где ты просишь главного агента использовать субагента для поиска информации.
- Скрин результата, где видно, что главный агент вызвал субагента и вернул итоговый агрегированный ответ.

Вариант 2 (Продвинутый — адаптируй под свою задачу) ☞

Спроектируй и запусти **мультиагентный пайплайн** для одной из своих реальных рабочих задач.

Что сделать:

1. Выбери задачу, которую ты сейчас решаешь при помощи нескольких последовательных запросов к одному агенту
2. Ответь на вопросы из урока: что на выходе, какие этапы, какие роли, как передаются результаты, как проверяется качество
3. Создай минимум **двух субагентов** с разными ролями
4. Напиши или адаптируй скилл-оркестратор, который описывает порядок работы субагентов (параллельный, последовательный или смешанный)
5. Запусти пайплайн, проверь результат, зафиксируй, что сработало, а что нужно доработать

**Ожидаемый результат:** работающая мультиагентная система под твою задачу. Субагенты созданы, пайплайн запущен, результат получен — пусть даже с шероховатостями, которые ты уже знаешь, как итерационно исправить.

**Что приложить к домашнему заданию:**

- Скрин папки `.claude/agents/`, где видны минимум два созданных субагента.
- Скрин файлов субагентов с их ролями, инструментами и frontmatter.
- Скрин скилла-оркестратора или инструкции, где описан порядок работы пайплайна.
- Скрин запуска пайплайна в Claude Code.
- Скрин результата работы пайплайна: итоговый файл, ответ, публикация, отчёт или другой артефакт.
- Короткий текстовый комментарий: что сработало хорошо, что нужно доработать и какую следующую итерацию ты бы сделал.

## УРОК 11

### [Практический курс по вайб-кодингу на Claude Code](#) [Модуль 4: Запуск проектов, мультиагентность и монетизация навыка](#)

CLm02 Деплой проекта: сервер, SSH, SSL и базовая безопасность

[Предыдущий урок](#)    Есть задание

[Следующий урок](#)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)



Как проходить уроки эффективно: доступы, оплата и лимиты

- 1** Доступ к мировым технологиям. Мы показываем лучшие мировые нейросети, но доступ к некоторым из них из РФ и/или других стран может требовать средств защищенного доступа. Мы действуем в рамках закона и НЕ консультируем по настройке соединения, но показываем эти сервисы, потому что они — лидеры рынка. Разобраться с доступом к ним — значит получить конкурентное преимущество.
- 2** Весь курс можно пройти на бесплатных версиях инструментов. Мы показываем платные PRO-функции для ознакомления, чтобы ты мог(ла) оценить их потенциал и осознанно решить, нужны ли они тебе в будущем.
- 3** Умное управление лимитами. Бесплатные попытки ограничены, поэтому регистрируйся в сервисе только перед выполнением ДЗ. Если лимиты закончились, а попрактиковаться еще хочется — часто помогает регистрация нового аккаунта на другую почту.
- 4** Сначала смотри, потом делай. Сначала посмотри урок полностью, чтобы понять логику, и только потом трать генерации. Это сэкономит твоё время и попытки.

Так ты освоишь самые мощные инструменты бесплатно и без лишних нервов! 🚀

---

#### Результат дня

- ✅ Понимаем, зачем нужен деплой и чем локальный запуск отличается от публикации в интернет
  - ✅ Знаем три типа платформ для размещения проекта: managed-платформы, хостинг и VPS — и умеем выбирать подходящий
  - ✅ Умеем загружать статический сайт на хостинг Beget через файловый менеджер
  - ✅ Умеем покупать домен, настраивать DNS-записи и привязывать домен к хостингу
  - ✅ Знаем, как выпустить SSL/TLS-сертификат и перевести сайт на HTTPS, и применяем базовые принципы безопасности при деплое
- ▶️ Время чтения и просмотра: ~60 минут
- 🕒 Время выполнения задания: ~60 минут



Под каждым видео в уроке есть текстовая расшифровка, чтобы у тебя была возможность лучше запомнить новые термины, прочитать пояснения и инструкции.

Чтобы прочитать текст, нажимай на него. Текст скрыт в строках подчёркнутых пунктирной линией со значком "↵" в конце строки.

---

## Теория на сегодня



**Важно:** этот урок знакомит вас с разными вариантами деплоя проекта. Вам необязательно повторять все действия эксперта и использовать именно те ресурсы, которые показаны в демонстрации.

Работайте со своим проектом и выбирайте подходящий способ деплоя под вашу задачу. По ходу работы вы можете обращаться к Claude Code: он поможет настроить инструменты, связать их между собой, разобраться в ошибках и пройти процесс шаг за шагом.

Если вы не планируете деплоить проект прямо сейчас, можно изучить материал теоретически и выбрать **первый вариант домашнего задания**.



### Лайфхак: ИИ может помочь с деплоем на VPS

Если вы купили облачный сервер, например в Timeweb Cloud, можно передать Claude Code данные для подключения: IP-адрес, SSH-логин и описание проекта. Он поможет подобрать команды, установить зависимости и опубликовать проект на сервере.

Используйте это только для учебного сервера без реальных данных. Не передавайте доступы от важных проектов, не загружайте секретные ключи без понимания и после эксперимента смените пароль или пересоздайте сервер.

## 1. Деплой проекта: сервер, SSH, SSL, базовая безопасность

Зачем нужен деплой ↵



**Деплой** — это момент, когда проект переносится из локальной среды разработки в интернет и становится доступен всем пользователям.

Локальный запуск удобен для разработки: мы пишем код, проверяем изменения, исправляем ошибки, экспериментируем. Сайт работает по адресу *localhost* и виден только нам на нашем компьютере.



**Задеплоенный проект** — это уже сервис, размещённый на удалённом сервере, к которому могут обращаться другие люди.

Деплой — это запуск проекта в среде, где на него может посмотреть весь интернет.



**Сервер** — это условно отдельный компьютер, который постоянно включён, подключён к интернету и готов обслуживать запросы пользователей: принимать их и отдавать ответы. У пользователя есть адрес, по которому он открывает сайт, API-сервис, бота, ИИ-продукт или что-то ещё.

Три варианта платформ для размещения проекта ↘

Рассмотрим три основных варианта того, где может жить проект.

**1. Managed-платформы:** [Vercel](#), [Railway](#), [Netlify](#) и др.

Это удобные сервисы, которые работают в связке с [GitHub](#) и позволяют быстро разместить код проекта без дополнительных настроек: без ручной установки SSL/TLS-сертификатов и часто без отдельной настройки домена. Загрузили — работает. Как правило, есть бесплатный тир.

- [Vercel](#) — оптимален для фронтенда и проектов на фреймворке **Next.js**. Бэкенды тоже возможны, но менее удобны.
- [Railway](#) — более универсальный вариант: базы данных, бэкенды, фронтенды.
- [Netlify](#) — удобен для статических сайтов, лендингов и фронтенд-проектов.

**2. Обычный хостинг**

Удобен для лендингов, визиток, сайтов на **Vanilla HTML/CSS/JavaScript**, а также для сайтов на **PHP** и **CMS**: например, [WordPress](#). Подходит для простых сайтов без сложной серверной логики.

**3. VPS (Virtual Private Server)**

Отдельный виртуальный сервер у провайдера, который арендуется целиком. Значительно более широкие возможности. Используется для:

- AI-агентов и автоматизаций;
- сервисов с хранением и обработкой данных;
- ботов и постоянных бэкендов с Docker;
- всего, что должно работать непрерывно или постоянно ожидать входящих данных.

Для VPS можно рассмотреть, например, [Selectel](#), [Timeweb Cloud](#), [REG.RU](#) или другие аналогичные сервисы.



Сориентируйся, что ты создавал(а) в рамках курса, есть ли готовый продукт, который хочется опубликовать, и выбери подходящий тип платформы.

**Примеры соответствия проектов платформам:**

Что размещаем	Куда депл
Статический сайт, лендинг, фронтенд	Хостинг
Next.js-проект	Vercel
Небольшой бэкенд, API, демо-сервис	Managed

Что размещаем	Куда депл
CMS и PHP-сайт	Хостинг
Бот, постоянный бэкенд, Docker	Managed
AI-агенты: n8n, OpenWebUI и др.	VPS

Сравнение сервисов для деплоя и дополнительные варианты

При выборе платформы важно учитывать не только тип проекта, но и доступность оплаты. Часть зарубежных managed-платформ удобна технически, но может быть недоступна для оплаты российской картой. Поэтому ниже — ориентиры: российские сервисы и зарубежные варианты, которые можно рассматривать при наличии зарубежной карты.

Сервис	Тип платформы	Для каких проектов подходит	Оплата	Примерная стоимость
<a href="#">Beget</a>	Хостинг + VPS	Статические сайты, PHP/CMS, простые сайты, VPS для бэкенда	Российская карта	Виртуальный хостинг — день, VPS — от 0,13 € в
<a href="#">Timeweb Cloud</a>	VPS/VDS, облако	Боты, бэкенды, Docker, небольшие сервисы	Российская карта	Зависит от конфигурации
<a href="#">Selectel</a>	VPS/VDS, облачная инфраструктура	Продакшен-сервисы, корпоративные проекты, базы данных, сложные бэкенды	Российская карта / договор	Почасовая оплата, итог конфигурации
<a href="#">REG.RU</a>	Хостинг + VPS	Домены, сайты, WordPress, VPS, небольшие серверные проекты	Российская карта	Есть Free Tier VPS на 6 м vCPU, 1 ГБ RAM, 10 ГБ S VPS — от 19 ₽/сутки
<a href="#">SpaceWeb</a>	Хостинг + VPS	Лендинги, сайты на CMS, простые проекты, VPS	Российская карта	Хостинг — от 119 ₽/мес. ₽/мес.
<a href="#">Amvera Cloud</a>	Managed-платформа / PaaS	Python, Node.js, Java, Go, боты, API, Docker-проекты	Российская карта	В открытых обзорах ука ₽/мес. за приложение
<a href="#">Yandex Cloud</a>	Облачная инфраструктура	VPS, серверные приложения, базы данных, хранилища, serverless	Российская карта / договор	Виртуальная машина м начинаться примерно о точная цена считается в



Сервис	Тип платформы	Для каких проектов подходит	Оплата	Примерная стоимость
<a href="#">Vercel</a>	Managed-платформа	Next.js, фронтенд, статические сайты, демо-проекты	Зарубежная карта	Hobby — бесплатно, Pro — \$20/мес.
<a href="#">Netlify</a>	Managed-платформа	Статические сайты, фронтенд, лендинги, JAMstack	Зарубежная карта	Free — \$0, Personal — \$20/мес.
<a href="#">Railway</a>	Managed-платформа / PaaS	Бэкенды, API, базы данных, боты, fullstack-проекты	Зарубежная карта	Pro — \$20 minimum usage, включены \$20 usage credits
<a href="#">Render</a>	Managed-платформа / PaaS	Web-сервисы, API, фоновые воркеры, бэкенды, агенты	Зарубежная карта	Hobby — \$0 + compute, \$25/мес. + compute

Как выбирать платформу ↴

Ситуация

Нужно просто показать статический сайт или лендинг
Проект на Next.js
Нужен бот, API или небольшой бэкенд
Нужен Docker и полный контроль
Нужна российская оплата
Есть зарубежная карта и нужен быстрый деплой через GitHub
Учебный проект без сложной серверной логики
AI-агент, n8n, OpenWebUI, постоянный сервис

Термины: SSH и SSL/TLS ↴

**SSH (Secure Shell)** — защищённый способ войти на удалённый сервер и управлять им через терминал. Используется при работе с VPS. На обычном хостинге SSH, как правило, не нужен — там

взаимодействие происходит через панель управления. На managed-платформах SSH тоже не нужен: всё загружается через GitHub.

**SSL/TLS-сертификат** — когда говорят «поставить SSL», имеют в виду именно **TLS-сертификат**, который обеспечивает работу сайта по протоколу **HTTPS**.

**HTTPS** — защищённый протокол связи между браузером пользователя и сайтом. Без HTTPS браузеры показывают предупреждение «Не защищено». Поэтому перед тем, как отправлять сайт пользователям, необходимо настроить TLS-сертификат.

## 2. Загружаем статический сайт на хостинг Beget

Шаги деплоя: загружаем статический сайт на хостинг Beget ↘

Мы идём по пути **обычного хостинга**. В качестве провайдера используем **Beget**.

Базовые принципы, которые мы разберём в этом уроке, пригодятся и при работе с **VPS**: вы поймёте, как устроена загрузка проекта, где должны лежать файлы сайта и почему важно не публиковать лишние данные.

Шаг 1. Зарегистрируйтесь в Beget

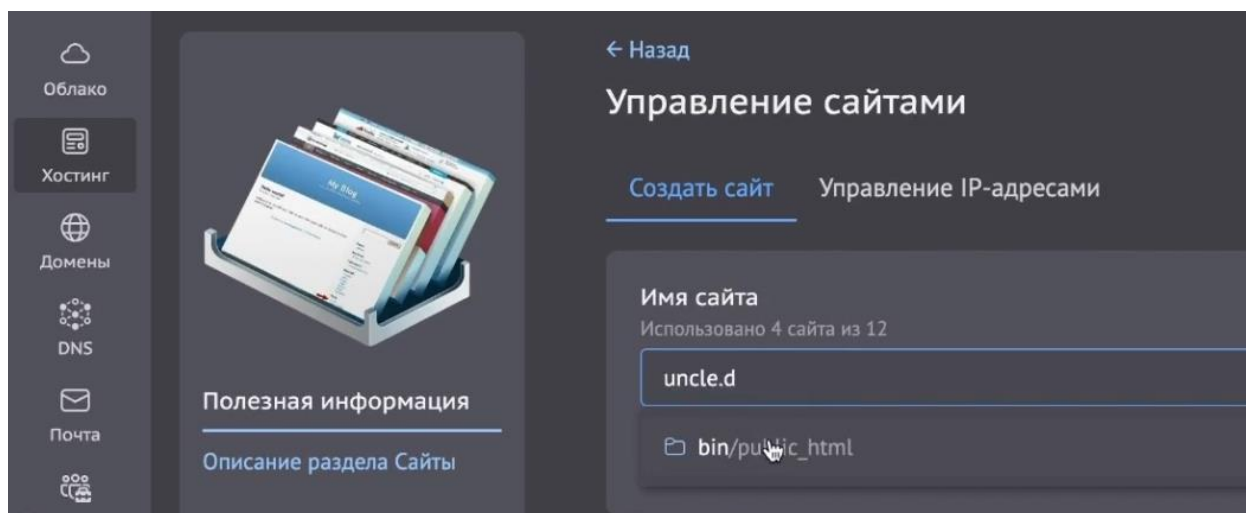
Для регистрации понадобятся:

- мобильный номер;
- email.

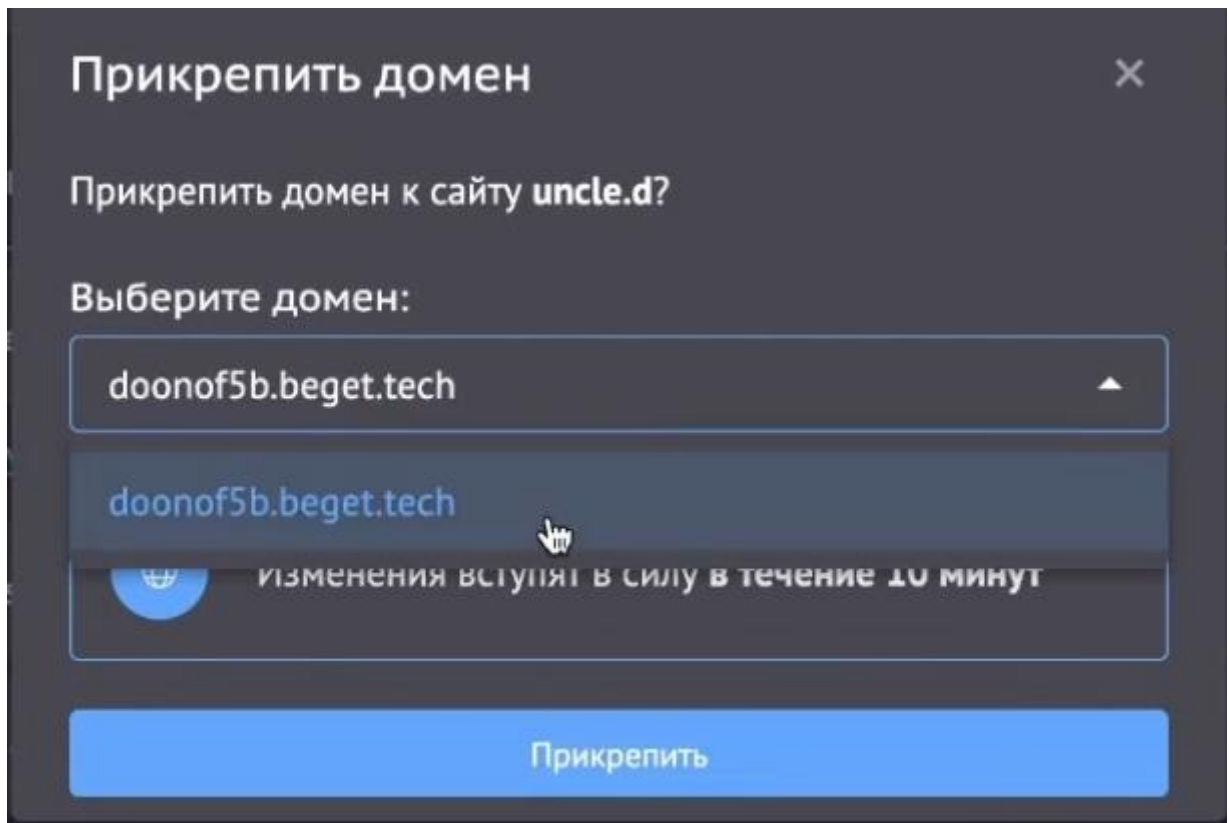
После регистрации логин и пароль придут на email. **Не теряйте это письмо**: данные понадобятся для входа в панель управления.

Шаг 2. Создайте сайт в панели управления

1. Перейдите в раздел «Сайты».



2. Создайте новый сайт с нужным именем.
3. После создания привяжите к нему **технический домен**.



Технический домен нужен, чтобы сразу проверить работу сайта без покупки собственного домена.

Шаг 3. Подготовьте файлы проекта

Откройте папку с проектом и проверьте, что в ней нет ничего лишнего.

В папке не должно быть:

- README-файлов;
- внутренней документации;
- .env файлов с ключами;
- тестовых или служебных файлов, которые не нужны для работы сайта.

Оставьте только файлы, необходимые для запуска сайта. После этого архивируйте содержимое папки в **ZIP**.

⚠ Всё, что попадёт в папку **public\_html**, сразу станет доступно всему интернету. Не кладите туда конфиденциальные файлы, секретные ключи и служебную документацию.

Шаг 4. Загрузите проект в файловый менеджер

1. Перейдите по пути: «Управление» → «Перейти к файлам сайта».
2. Откройте папку **public\_html**.
3. Нажмите «Загрузить файлы».
4. Выберите подготовленный **ZIP-архив**.
5. После загрузки нажмите «Распаковать архив».
6. Путь распаковки оставьте предложенный системой.

7. После распаковки удалите ZIP-архив.

💡 Файловый менеджер также можно открыть напрямую через панель управления: «Хостинг» → «Файловый менеджер».

### Шаг 5. Проверьте сайт 📄

Откройте технический домен в браузере.

Проверьте, что:

- сайт открывается;
- картинки отображаются;
- анимации работают;
- кнопки и ссылки кликаются;
- структура страницы не сломалась после загрузки.

⚠️ **Ключевое правило:** файл `index.html` должен лежать **в корне папки `public_html`**. Если он окажется во вложенной папке, сайт не откроется корректно.

Типичные ошибки при деплое 📄

- **Неправильное расположение `index.html`** — файл должен быть в корне ``public_html``, а не во вложенной папке внутри неё.
- **Некорректные пути к файлам** — при локальной разработке пути могут отличаться от тех, что нужны на сервере. Пути должны быть написаны относительно корневой директории. Если возникла такая ошибка — попроси Claude исправить пути в коде.
- **Конфликт с существующим `index.html`** — если в папке уже лежит старый ``index.html``, он может конфликтовать с новым. Убедись, что файл один.
- **Ошибки привязки домена или DNS** — рассмотрим в следующих видео.

**Обновление сайта в будущем:** если нужно внести изменения, самый простой способ — снова заархивировать все файлы, удалить старое содержимое из ``public_html``, загрузить новый архив и распаковать его. Если менялся только один файл — можно перенести только его.

### 3. Покупка домена на стороннем регистраторе

Покупка домена 📄

Домен можно купить как внутри Beget, так и у стороннего регистратора — например, на [reg.ru](https://reg.ru) или другом аналогичном сервисе. Это полезно знать, если домен уже куплен в другом месте или если хочется сравнить цены.

Процесс покупки:

1. Вводим желаемое имя домена и нажимаем «Подобрать».

# Зарегистрировать домен

uncle-d.ru

Подобрать

2. Выбираем подходящий вариант из предложенных.

uncle-d.ru

Выгоднее с Рег.решением → 226 169 ₽

Подобрать

uncle-d.online

Вместе с .ru за 1 рубль 4067 249 ₽

Подобрать

uncle-d.com

Домен занят Как купить занятый домен?

Подобрать

uncled.com

Домен занят Как купить занятый домен?

Подобрать

uncle-d.shop

Подобрать

uncled.shop

Подобрать

3. Убираем лишние дополнительные услуги, которые предлагает регистратор при оформлении заказа, чтобы избежать неожиданных списаний в будущем.
4. Регистрируемся или входим в аккаунт.
5. Заполняем контактную информацию — при покупке домена важно указывать реальные данные, так как домен оформляется на владельца.
6. Оплачиваем удобным способом.



Если домен уже куплен на другом сервисе — его можно перенести в Beget через раздел «Домены» → «Перенос домена». Либо оставить у текущего регистратора и просто настроить DNS-записи, указывающие на Beget.

## 4. Настройка DNS: связываем домен с хостингом

Добавление домена в Beget и настройка DNS

После покупки домена нам нужно связать его с хостингом. Если домен куплен у стороннего регистратора, это делается через **DNS-записи**.

### Шаг 1. Добавляем домен в Beget

В разделе «Домены и поддомены» вводим имя домена и добавляем его в список. После этого переходим в «Редактировать DNS» — здесь Beget покажет **IP-адрес**, который нужно прописать у регистратора.



## Шаг 2. Добавляем DNS-записи у регистратора

Переходим в настройки домена на сайте регистратора (например, reg.ru): раздел «DNS-серверы и управление зоной» → «Изменить».

Добавляем две **A-записи**:

- Тип: `A`, значение: IP-адрес из Beget, хост: `@`
- Тип: `A`, значение: тот же IP-адрес, хост: `www`

Старые записи, которые там были, удаляем — они больше не нужны.

## Шаг 3. Привязываем домен к сайту в Beget

В разделе «Сайты» находим нужный сайт, нажимаем «Прикрепить домен» и выбираем только что добавленный домен. Изменения вступают в силу в течение 10 минут — иногда нужно просто подождать.



DNS-записи распространяются не мгновенно. Если сайт не открывается сразу — подожди, почисти кэш браузера или проверь в другом браузере. Также убедись, что email при регистрации у регистратора подтверждён — без этого домен может не активироваться.

При помощи DNS мы связали провайдера домена с хостингом.

## 5. Привязка домена, проверка сайта и выпуск SSL-сертификата

Финальная проверка привязки домена ☝

После того как DNS-записи распространились, проверяем, что домен привязан к нужному сайту. Если что-то пошло не так (например, создался лишний сайт), домен всегда можно открепить и прикрепить заново к нужной папке — домен никуда не пропадёт, он зафиксирован в Beget.

Когда сайт успешно открывается по домену — переходим к следующему шагу.

Выпуск SSL/TLS-сертификата ☝

Сайт открывается, но соединение пока работает по **HTTP** — браузер показывает неприятный восклицательный знак «Не защищено». Нам нужно перевести его на **HTTPS**.

**Как выпустить сертификат в Beget:**

Переходим: «Домены» → «Управление SSL-сертификатами» → выбираем бесплатный **Let's Encrypt** → нажимаем «Установить».

Заявка отправляется автоматически. Выпуск сертификата для нового домена может занять **от нескольких часов до двух дней**. Информация о статусе придёт на email.



Let's Encrypt — бесплатный TLS-сертификат, которого полностью достаточно для обычного сайта. Он доступен на любом хостинге.



Не отправляй сайт пользователям до тех пор, пока SSL-сертификат не выпущен и сайт не открывается по HTTPS.

Итог: что мы сделали ☞

Подведём итог всего процесса деплоя:

1. **Создали сайт** в Beget — папка с именем сайта, внутри которой находится `public\_html` со всеми файлами проекта.
2. **Загрузили файлы** через файловый менеджер: архивировали, загрузили, распаковали.
3. **Купили домен** — в Beget или у стороннего регистратора.
4. **Настроили DNS** — добавили A-записи с IP-адресом Beget на стороне регистратора домена.
5. **Привязали домен** к сайту в Beget.
6. **Выпустили SSL-сертификат** через Let's Encrypt — ждём активации.

После активации сертификата восклицательный знак исчезнет и сайт будет работать по защищённому протоколу HTTPS.

## 6. Базовые принципы безопасности при деплое

Что не должно попасть в public\_html ☞

В папку `public\_html` кладём **только то, что должен увидеть браузер пользователя**:

- ❌ README и документация
- ❌ Служебные и системные файлы
- ❌ `.env` и `.env.local` — файлы с секретными ключами и токенами
- ❌ Системные промпты агентов

Серверная логика (например, PHP-бэкенд) может находиться в других папках за пределами `public\_html`. В публичной папке — исключительно фронтенд.



Всё, что лежит в `public\_html`, немедленно становится доступным всему интернету.

Защита аккаунтов и конфиденциальных данных ☞

- **Защищаем паролем** доступ к панели Beget и к аккаунту регистратора домена. Если доступна двухфакторная аутентификация — включаем её.
- **Не отправляем в чат с ИИ** пароли, секретные ключи, платёжные данные и любую приватную информацию. Как только данные отправлены — они считаются скомпрометированными. В ИИ отправляем только контекст, документацию и общую информацию о проекте.

- **Понимаем, куда уходят данные форм.** Если на лендинге есть форма заявки, нужно чётко понимать, кто и как получает эти данные. Если форма ещё не настроена — лучше поставить ссылку на соцсеть или мессенджер, чем оставлять заглушку.

Хорошие привычки после деплоя ☞

- **Делаем бэкапы** проекта при каждом обновлении. Базовый вариант — создавать папки с версиями. Более профессиональный — использовать **GitHub**: завести аккаунт, установить **Git** и версионировать проект правильно.
- **Проверяем сайт глазами пользователя** после каждой публикации: все страницы, формы, ссылки, изображения. Убеждаемся, что не выложили лишние файлы.
- **Обязательно подключаем HTTPS** перед тем, как делиться сайтом с пользователями.



Чтобы развивать сайт дальше, можно спросить Claude Code: что можно улучшить во фронтенде, нужен ли бэкенд и как его написать, какую дополнительную документацию (например, по Beget) стоит добавить в контекст. Главное — не передавать секретные ключи и пароли.

### Дополнительные материалы

Ссылки на сервисы из урока ☞

- **Beget** (beget.com) — российский хостинг-провайдер. Используем для размещения статического сайта и управления доменом.
- **Reg.ru** (reg.ru) — регистратор доменов. Используем для покупки домена и настройки DNS-записей.
- **Vercel** (vercel.com) — managed-платформа для деплоя фронтенда и Next.js-проектов.
- **Railway** (railway.app) — managed-платформа для деплоя бэкендов, баз данных и фронтендов.
- **Netlify** (netlify.com) — managed-платформа для деплоя статических сайтов и фронтенда.
- **GitHub** (github.com) — сервис для хранения кода и версионирования проектов. Рекомендуется для бэкапов и профессиональной работы с кодом.
- **Let's Encrypt** — бесплатный центр выдачи TLS-сертификатов. Доступен через панель управления большинства хостингов.

Словарь занятия ☞

- **Деплой** — процесс переноса проекта с локального компьютера на удалённый сервер, после которого сайт или сервис становится доступен всем пользователям интернета.
- **Сервер** — компьютер, постоянно подключённый к интернету и готовый обслуживать запросы пользователей.
- **Localhost** — локальный адрес, по которому проект открывается только на вашем компьютере во время разработки.
- **Хостинг** — услуга аренды места на сервере для размещения файлов сайта. Подходит для статических сайтов, лендингов, PHP-сайтов и CMS.



- **VPS (Virtual Private Server)** — виртуальный выделенный сервер, который арендуется целиком. Подходит для сложных сервисов, ботов, AI-агентов и всего, что требует постоянной работы.
- **Managed-платформа** — облачный сервис (Vercel, Railway, Netlify), который берёт на себя инфраструктурные задачи: деплой, масштабирование, HTTPS. Обычно работает в связке с GitHub.
- **public\_html** — корневая публичная папка сайта на хостинге. Всё, что в ней лежит, доступно пользователям интернета.
- **SSH (Secure Shell)** — защищённый протокол для подключения к удалённому серверу и управления им через командную строку. Используется при работе с VPS.
- **SSL/TLS-сертификат** — цифровой сертификат, обеспечивающий шифрование соединения между браузером и сервером.
- **HTTP / HTTPS** — протоколы передачи данных в интернете. HTTPS — защищённая версия, работающая с TLS-сертификатом.
- **DNS (Domain Name System)** — система, которая связывает доменное имя (например, *mysite.ru*) с IP-адресом сервера, где находится сайт.
- **A-запись** — тип DNS-записи, которая указывает, на какой IP-адрес должен резолвиться домен.
- **Let's Encrypt** — бесплатный центр сертификации, выдающий TLS-сертификаты. Доступен на большинстве хостингов.
- **GitHub** — сервис для хранения кода с системой версионирования Git. Используется для бэкапов и командной разработки.
- **.env** — файл с переменными окружения, в котором хранятся секретные ключи и токены. Никогда не должен попадать в публичную папку или в чат с ИИ.
- **CMS (Content Management System)** — система управления контентом (например, WordPress). Позволяет управлять сайтом без написания кода.

### Домашнее задание

Предлагаем три варианта заданий. Первый поможет познакомиться с логикой деплоя без дополнительных расходов, второй — повторить практику за экспертом и опубликовать простой сайт, а третий — адаптировать деплой под свой проект и продумать его дальнейшее развитие.



Выбери только один вариант домашнего задания и выполни его сегодня.

Другой вариант, если захочешь, можешь проработать позже самостоятельно для закрепления материала.

#### Вариант 1 (Базовый — ознакомительный) ↘

Разбери процесс деплоя статического HTML-сайта на примере Beget и подготовь проект к публикации.

В этом варианте не нужно оплачивать хостинг, покупать домен или выпускать SSL-сертификат. Задача — понять, какие шаги нужны для публикации сайта в интернете, и подготовить файлы так, будто ты готовишься к реальному деплою.

**Что нужно сделать:**

1. Подготовь файлы проекта: убедись, что в папке нет ничего лишнего — README, .env, служебной документации, файлов с ключами и паролями.
2. Проверь, что главный файл сайта называется index.html и лежит в корне папки проекта.
3. Заархивируй содержимое папки проекта в ZIP.
4. Изучи интерфейс Beget или другого хостинга по материалам урока: где создаётся сайт, где находится файловый менеджер, куда загружается архив, зачем нужна папка public\_html.
5. Опиши, какие действия ты выполнил(а) бы при реальном деплое:
  - зарегистрироваться на хостинге;
  - создать сайт;
  - загрузить ZIP-архив в public\_html;
  - распаковать архив;
  - проверить сайт по техническому домену;
  - при необходимости подключить домен;
  - выпустить SSL-сертификат.
6. Проверь проект локально в браузере: открой index.html и убедись, что текст, изображения, кнопки и ссылки отображаются корректно.

**Что приложить в отчёт:**

- скрин папки проекта с файлом index.html;
- скрин ZIP-архива с файлами сайта;
- скрин открытого сайта локально в браузере;
- короткое описание: какие шаги нужны, чтобы этот проект стал доступен в интернете.

**Результат:** ты понимаешь логику деплоя, знаешь, какие файлы можно загружать на хостинг, где должен лежать index.html, зачем нужна папка public\_html и что потребуется для реальной публикации сайта.

Вариант 2 (Продвинутый — повтори за экспертом) ☑

Задеплой HTML-лендинг или визитку на хостинг Beget.

Этот вариант подойдёт тем, кто готов зарегистрироваться на хостинге и опубликовать сайт в интернете.

**Что нужно сделать:**

1. Подготовь файлы проекта: убедись, что в папке нет ничего лишнего — README, .env, документации, файлов с ключами и паролями. Заархивируй содержимое папки в ZIP.

2. Зарегистрируйся в Beget, создай сайт, перейди в файловый менеджер и загрузи архив в папку `public_html`. Распакуй архив.
3. Открой сайт по техническому домену Beget и убедись, что все элементы — картинки, кнопки, ссылки, анимации — работают корректно.
4. Если у тебя уже есть домен, привяжи его: добавь домен в Beget, пропиши нужные DNS-записи у регистратора домена и прикрепи домен к сайту.
5. Выпусти бесплатный SSL-сертификат через Let's Encrypt в разделе управления SSL-сертификатами. Дождись активации и проверь, что сайт открывается по HTTPS.

**Что приложить в отчёт:**

- ссылку на опубликованный сайт;
- скрин файлового менеджера с содержимым `public_html`;
- скрин сайта, открытого по техническому домену или своему домену;
- скрин HTTPS-версии сайта или активного SSL-сертификата;
- короткий комментарий: что получилось, какие сложности возникли, как ты их решил(а).

**Результат:** работающий сайт в интернете, открывающийся по техническому домену или собственному домену. При подключении SSL сайт открывается по защищённому протоколу HTTPS.

Вариант 3 (Продвинутый — адаптируй под свою задачу) ☝

Возьми любой свой проект, созданный в рамках курса или самостоятельно, и опубликуй его в интернете на подходящей платформе.

**Что нужно сделать:**

1. Определи тип своего проекта: статический сайт, Next.js-приложение, бэкенд-сервис, бот, API или другой формат.
2. Выбери подходящую платформу для деплоя: Beget, Vercel, Railway, Render, Amvera, VPS или другой сервис.
3. Задеплой проект, следуя логике урока: загрузи файлы, подключи репозиторий или настрой сервер, проверь запуск проекта.
4. Если нужно, настрой домен, DNS-записи и SSL-сертификат.
5. После публикации спроси **Claude Code**, как можно развить проект дальше: улучшить интерфейс, добавить бэкенд, подключить базу данных, авторизацию, формы, бота или другие функции. Используй документацию выбранного хостинга как контекст для ИИ.
6. Проверь проект глазами пользователя: все страницы, формы, ссылки, изображения и основные сценарии работы.

**Что приложить в отчёт:**

- ссылку на опубликованный проект;
- скрин выбранной платформы или панели деплоя;
- скрин работающего проекта в браузере;
- скрин или текст ответа Claude Code с идеями развития проекта;

- короткое описание: какую платформу ты выбрал(а), почему она подходит именно для этого проекта.

**Результат:** опубликованный в интернете проект и конкретный план его дальнейшего развития, сформированный с помощью Claude Code.

### Задание

Предлагаем три варианта заданий. Первый поможет познакомиться с логикой деплоя без дополнительных расходов, второй — повторить практику за экспертом и опубликовать простой сайт, а третий — адаптировать деплой под свой проект и продумать его дальнейшее развитие.



Выбери только один вариант домашнего задания и выполни его сегодня.

Другой вариант, если захочешь, можешь проработать позже самостоятельно для закрепления материала.

Вариант 1 (Базовый — ознакомительный) ▾

Разбери процесс деплоя статического HTML-сайта на примере Beget и подготовь проект к публикации.

В этом варианте не нужно оплачивать хостинг, покупать домен или выпускать SSL-сертификат. Задача — понять, какие шаги нужны для публикации сайта в интернете, и подготовить файлы так, будто ты готовишься к реальному деплою.

#### Что нужно сделать:

1. Подготовь файлы проекта: убедись, что в папке нет ничего лишнего — README, .env, служебной документации, файлов с ключами и паролями.
2. Проверь, что главный файл сайта называется index.html и лежит в корне папки проекта.
3. Заархивируй содержимое папки проекта в ZIP.
4. Изучи интерфейс Beget или другого хостинга по материалам урока: где создаётся сайт, где находится файловый менеджер, куда загружается архив, зачем нужна папка public\_html.
5. Опиши, какие действия ты выполнил(а) бы при реальном деплое:
  - зарегистрироваться на хостинге;
  - создать сайт;
  - загрузить ZIP-архив в public\_html;
  - распаковать архив;
  - проверить сайт по техническому домену;
  - при необходимости подключить домен;
  - выпустить SSL-сертификат.
6. Проверь проект локально в браузере: открой index.html и убедись, что текст, изображения, кнопки и ссылки отображаются корректно.

#### Что приложить в отчёт:

- скрин папки проекта с файлом index.html;
- скрин ZIP-архива с файлами сайта;
- скрин открытого сайта локально в браузере;
- короткое описание: какие шаги нужны, чтобы этот проект стал доступен в интернете.

**Результат:** ты понимаешь логику деплоя, знаешь, какие файлы можно загружать на хостинг, где должен лежать index.html, зачем нужна папка public\_html и что потребуется для реальной публикации сайта.

Вариант 2 (Продвинутый — повтори за экспертом) ☞

Задеплой HTML-лендинг или визитку на хостинг Beget.

Этот вариант подойдёт тем, кто готов зарегистрироваться на хостинге и опубликовать сайт в интернете.

**Что нужно сделать:**

1. Подготовь файлы проекта: убедись, что в папке нет ничего лишнего — README, .env, документации, файлов с ключами и паролями. Заархивируй содержимое папки в ZIP.
2. Зарегистрируйся в Beget, создай сайт, перейди в файловый менеджер и загрузи архив в папку public\_html. Распакуй архив.
3. Открой сайт по техническому домену Beget и убедись, что все элементы — картинки, кнопки, ссылки, анимации — работают корректно.
4. Если у тебя уже есть домен, привяжи его: добавь домен в Beget, пропиши нужные DNS-записи у регистратора домена и прикрепи домен к сайту.
5. Выпусти бесплатный SSL-сертификат через Let's Encrypt в разделе управления SSL-сертификатами. Дождись активации и проверь, что сайт открывается по HTTPS.

**Что приложить в отчёт:**

- ссылку на опубликованный сайт;
- скрин файлового менеджера с содержимым public\_html;
- скрин сайта, открытого по техническому домену или своему домену;
- скрин HTTPS-версии сайта или активного SSL-сертификата;
- короткий комментарий: что получилось, какие сложности возникли, как ты их решил(а).

**Результат:** работающий сайт в интернете, открывающийся по техническому домену или собственному домену. При подключении SSL сайт открывается по защищённому протоколу HTTPS.

Вариант 3 (Продвинутый — адаптируй под свою задачу) ☞

Возьми любой свой проект, созданный в рамках курса или самостоятельно, и опубликуй его в интернете на подходящей платформе.

**Что нужно сделать:**

1. Определи тип своего проекта: статический сайт, Next.js-приложение, бэкенд-сервис, бот, API или другой формат.

2. Выбери подходящую платформу для деплоя: Beget, Vercel, Railway, Render, Amvera, VPS или другой сервис.
3. Задеплой проект, следуя логике урока: загрузи файлы, подключи репозиторий или настрой сервер, проверь запуск проекта.
4. Если нужно, настрой домен, DNS-записи и SSL-сертификат.
5. После публикации спроси **Claude Code**, как можно развить проект дальше: улучшить интерфейс, добавить бэкенд, подключить базу данных, авторизацию, формы, бота или другие функции. Используй документацию выбранного хостинга как контекст для ИИ.
6. Проверь проект глазами пользователя: все страницы, формы, ссылки, изображения и основные сценарии работы.

#### Что приложить в отчёт:

- ссылку на опубликованный проект;
- скрин выбранной платформы или панели деплоя;
- скрин работающего проекта в браузере;
- скрин или текст ответа Claude Code с идеями развития проекта;
- короткое описание: какую платформу ты выбрал(а), почему она подходит именно для этого проекта.

**Результат:** опубликованный в интернете проект и конкретный план его дальнейшего развития, сформированный с помощью Claude Code.

## УРОК 12

[Практический курс по вайб-кодингу на Claude Code](#) [Модуль 4: Запуск проектов, мультиагентность и монетизация навыка](#)

CLm03 Монетизация навыка и финальный проект под личную задачу

[Предыдущий урок](#)    Есть задание

[Следующий урок](#)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)



Как проходить уроки эффективно: доступы, оплата и лимиты

**1** Доступ к мировым технологиям. Мы показываем лучшие мировые нейросети, но доступ к некоторым из них из РФ и/или других стран может требовать средств защищенного доступа. Мы действуем в рамках закона и НЕ консультируем по настройке соединения, но показываем эти

сервисы, потому что они — лидеры рынка. Разобраться с доступом к ним — значит получить конкурентное преимущество.

**2** Весь курс можно пройти на бесплатных версиях инструментов. Мы показываем платные PRO-функции для ознакомления, чтобы ты мог(ла) оценить их потенциал и осознанно решить, нужны ли они тебе в будущем.

**3** Умное управление лимитами. Бесплатные попытки ограничены, поэтому регистрируйся в сервисе только перед выполнением ДЗ. Если лимиты закончились, а попрактиковаться еще хочется — часто помогает регистрация нового аккаунта на другую почту.

**4** Сначала смотри, потом делай. Сначала посмотри урок полностью, чтобы понять логику, и только потом трать генерации. Это сэкономит твоё время и попытки.

Так ты освоишь самые мощные инструменты бесплатно и без лишних нервов! 🚀

---

## Результат дня

- ✅ Понимаешь, почему навык работы с ИИ-агентами и кодинг-инструментами можно монетизировать уже сейчас
- ✅ Знаешь шесть форматов услуг, которые можно предлагать на основе полученной базы
- ✅ Умеешь описывать свои решения на языке пользы — через проблему, решение и конкретный результат
- ✅ Знаешь критерии сильного финального проекта и можешь оценить свою работу по ним
- ✅ Выбрал(а) вектор развития и составил(а) маршрут на ближайшие 30 дней

🕒 Время чтения и просмотра: ~40 минут

🕒 Время выполнения задания: ~40 минут



Под каждым видео в уроке есть текстовая расшифровка, чтобы у тебя была возможность лучше запомнить новые термины, прочитать пояснения и инструкции.

Чтобы прочитать текст, нажимай на него. Текст скрыт в строках подчёркнутых пунктирной линией со значком "⏏" в конце строки.

---

## Теория на сегодня

### 1. Монетизация навыка, финальный проект и всё, что мы прошли за курс

Почему навык можно монетизировать ⏏

Посмотрим на реальную картину рынка. Более **6 миллиардов человек** на планете в принципе не используют искусственный интеллект. Чуть больше миллиарда используют бесплатные версии — **ChatGPT, Gemini** и подобные сервисы с крайне ограниченными возможностями. Совсем небольшая часть людей платит за подписку. И лишь около **0,04% населения** работает с так называемой **агентной архитектурой**.

Вы находитесь в числе тех, кто использует агентные решения. Мы с вами действительно в самом начале пути — и очень круто, что вы в это запрыгнули.

Это говорит об огромном количестве перспектив для тех, кто умеет работать с **ИИ-агентами**.



Ценность навыка появляется не тогда, когда ты просто говоришь «я умею использовать Claude Code». Ценность возникает, когда ты берёшь понятную задачу и доводишь её до рабочего состояния.

Правильные формулировки при позиционировании себя:

- «Довожу прикладную задачу до рабочего результата»
- «Автоматизирую повторяемую задачу для того, чтобы...»

Это **продуктовый подход**: находим боль — даём конкретное решение. На рынке много запросов, где не нужна большая команда разработки и долгий цикл. Малый бизнес сталкивается с понятными рабочими проблемами: нужен простой лендинг, внутренний помощник, автоматизация контента.

**Общая формула работы:**

1. Понять запрос
2. Собрать первый вариант — **MVP** (Minimal Viable Product)
3. Проверить и доработать
4. Запустить и объяснить пользу

## 2. Что можно предлагать после курса

Форматы услуг после курса ↴

Мы выделяем шесть направлений, в которых можно предлагать свои услуги:

1. **Сайт** — упаковка и привлечение. Внутренние сайты, лендинги, страницы услуг, портфолио. Людям нужно присутствие в цифровом пространстве — и вы можете закрыть эту потребность.
2. **Внутренний сервис** — панели, базы заявок, чек-листы, внутренние автоматизации для бизнеса, которые помогают ускорить рутинные рабочие процессы.
3. **ИИ-агент** — от простых чат-ботов и помощников по базе данных до сложных мультиагентных систем. Позиционирование: **архитектор агентных решений**.
4. **Автоматизация** — личные и бизнес-автоматизации, делегирование задач искусственному интеллекту.
5. **Контент** — контент-планы, генерация постов и медиафайлов, автоматические **контент-заводы** на заказ.
6. **Собственный продукт** — **SaaS** (Software as a Service). Продукт, который монетизируется через привлечение частных клиентов или решает повторяющуюся проблему бизнеса.



Во всех этих направлениях мы не продаём Claude Code, агентов или нейросеть. Мы продаём конкретный результат, который человек получит с помощью ИИ — и мы ему в этом помогаем.

**Пример:** один клиент — эксперт по английскому, автор онлайн-курсов или автосервис — может одновременно нуждаться в:



- сайте с формой заявки
- ассистенте для сбора лидов
- CRM-автоматизации для работы с клиентами
- контент-плане и автоматической публикации
- исследовательском агенте для анализа конкурентов
- внутренней автоматизации рутинных процессов

### 3. Как говорить о навыке на языке пользы

Как говорить о навыке на языке пользы ↴

Техническая база — это важно. Но когда показываешь проект заказчику, нужно мыслить иначе.

**Не так:** «Я подключил агентов» или «Я сделал сайт» — это просто техническая деталь, расплывчатая и непонятная клиенту.

**А вот так:**

- *«Отчёт собирается быстрее, меньше ручного копипаста, проще проверить результат»*
- *«У услуги появилась страница, куда можно приводить клиентов и получать заявки»*

Акцент на пользе делает ценность очевидной.

**Структура описания любого проекта:**

- **Проблема** — что было неудобно, долго, дорого или неструктурно; кто с этим сталкивался и как часто; почему это мешало работе
- **Решение** — что именно собрано в проекте; какой сценарий проходит пользователь на входе и что получает на выходе
- **Результат** — где экономия времени, где экономия денег, где снижается ручная рутина, повышается качество, скорость, предсказуемость

**Формула кейса:**

«Я сделал проект для [кого], который помогает решить [проблему]. Пользователь делает [действие], на выходе получает [результат]. Это полезно потому, что [польза]. Дальше проект можно развивать через [шаги].»



Используй эту формулу, чтобы оформить свои кейсы и уверенно идти к новым заказчикам.

Критерии сильного финального проекта ↴

Проект должен отвечать трём критериям:

- 1. Решает конкретную задачу.** Понятная проблема или потребность — личная, учебная, для работы, для фриланса, для потенциального клиента. Формулировка должна быть простой: кому помогает и зачем.
- 2. Демонстрирует сценарий.** Не просто «проект существует», а показано, что делает человек или система на входе и какой результат получается на выходе.

**3. Польза объясняется простыми словами.** Любой человек должен понять, почему это лучше, чем то, что было раньше.



Если к этому моменту у тебя что-то получилось не до конца или кажется неполноценным — это нормально. Мы осваивали базу. Попробуй пересмотреть нужные уроки и довести проект до рабочего состояния. Даже простая автоматизация или несколько субагентов — уже настоящий кейс.

**Примеры проектов, которые мы разобрали на курсе:**

- Лендинг для конкретной услуги с формой заявки
- Внутренний помощник, который собирает информацию и оформляет отчёт
- Агентная архитектура на локальной машине
- Контентный процесс с автоматической публикацией
- Любая небольшая автоматизация повторяемой ручной работы

Примеры реальных проектов ↴

**Проект 1: Лендинг.** Минималистичный лендинг с информацией об услугах, встроенным видео с YouTube-канала и блоком кейсов. В блоке кейсов показан конкретный результат: что было сделано, какой эффект получен, какие метрики улучшились. Форма обратной связи — ссылка в Telegram.

**Проект 2: Бот для автоматической публикации контента.** Заказчику была нужна система автоматического распространения контента. Решение: бот в Telegram, куда заказчик отправляет текстовый или видеоконтент. Бот добавляет хештеги, описания и названия, затем автоматически публикует на нужные площадки:

- Видео → TikTok, YouTube Shorts, Reels
- Текст → Telegram-канал, X, ВКонтакте

Сервис размещён на **Railway**, работает через **API** сервиса **Postis**, написан на **Python** с подключённой базой данных.



Оба проекта созданы исключительно при помощи кодинговых агентов — без глубокого знания языков программирования.

Что мы прошли за курс ↴

За время курса мы освоили следующие темы:

- **Терминал** — навигация внутри компьютера, работа с агентами через командную строку
- **Промптинг и системные инструкции** — программирование агента: правила, приоритеты, контекст проекта
- **Типы данных и файлов** — Markdown, CSV, Excel, JSON; визуализаторы кода ZED и Obsidian
- **Настройки кодинговых агентов** — Claude Code, settings.json, скрытые папки .claude, разница между глобальными и проектными инструкциями
- **Первый лендинг** — стек HTML, CSS, JavaScript; деплой и получение SSL-сертификата

- **Локальные модели** — работа с ИИ без интернета, локальные автоматизации
- **MCP и внешние инструменты** — стек инструментов для агентов: Playwright, Context7 и другие
- **Скиллы** — архитектурный подход к агентам, библиотеки готовых примеров
- **Контент-завод** — архитектура через JSON-файл и JSON-промптинг, автогенерация изображений и публикация
- **Мультиагентные системы** — делегирование задач через манифест JSON, чёткие input и output для агентов
- **Deploy и безопасность** — типы хостингов, managed-платформы, серверные решения

Ваш финальный проект — это точка опоры. Как только вы почувствуете, что качественно прокачали свою работу при помощи агентных систем, вы войдёте в те 0,04% людей, которые используют агентную архитектуру. И дальше всё пойдёт гораздо быстрее.

#### 4. Вектора развития после курса

Вектора развития после курса ↘

По направлениям:

- **Фронтенд** — сайты, лендинги, UI/UX-дизайн, визуальные решения
- **Бэкенд и сервисы** — автоматизации, инструменты для бизнеса, фреймворки и библиотеки
- **Агентные системы** — персональные агенты, агентные слои для бизнеса, архитектура мультиагентов
- **Автоматизация контента** — контент-заводы, системы публикации для блогеров и бизнеса
- **Собственный продукт / SaaS** — итерационный подход: MVP → фидбэк → доработка → снова аудитория

По области применения:

- **Корпоративная разработка (B2B)** — автоматизации и продукты для малого и среднего бизнеса
- **Карьера** — автоматизация операционных задач внутри своей деятельности, повышение продуктивности
- **Фриланс** — небольшие заказы: контент, микросервисы, Telegram-боты, картинки
- **Собственный продукт** — SaaS с итерационным циклом разработки

**Маршрут на ближайшие 30 дней — заполни для себя:**

1. Какой проект я беру за основу?
2. Тема, область, направление
3. Для кого он нужен и кому полезен?
4. Какой один сценарий внутри проекта должен работать?
5. Как я проверю результат?

6. Как это будет представлено в портфолио и как я это презентую клиенту, руководителю или команде?



Рекомендуем ответственно подойти к этому шагу и прямо сейчас дописать эти предложения. Это превратит курс в реальный результат.

### Дополнительные материалы

Ссылки на сервисы из урока ↘

- **Claude Code** — кодинговый агент от Anthropic для разработки через терминал
- **Railway** — managed-платформа для размещения сервисов и баз данных
- **Postis** — сервис для автоматической публикации контента на несколько площадок одновременно
- **Playwright** — MCP-инструмент для управления браузером из агента
- **Context7** — MCP-инструмент для работы с актуальной документацией библиотек
- **Obsidian** — визуализатор файлов и заметок, удобен для работы с Markdown
- **ZED** — редактор кода с удобным отображением файлов проекта
- **GPT Image** — инструмент генерации изображений от OpenAI

Словарь занятия ↘

- **Монетизация навыка** — превращение умения работать с инструментом в услугу или продукт, за который платят деньги
- **MVP (Minimal Viable Product)** — минимально жизнеспособный продукт; первая рабочая версия, собранная быстро для проверки гипотезы
- **SaaS (Software as a Service)** — программное обеспечение как услуга; продукт, за использование которого клиент платит регулярно
- **Агентная архитектура** — система, в которой ИИ-агент самостоятельно выполняет задачи, вызывает инструменты и делегирует подзадачи субагентам
- **Кодинг-скаффолд (coding scaffold)** — архитектура и инструменты вокруг ИИ, с помощью которых ведётся разработка
- **Продуктовый подход** — методология, при которой сначала находится боль (проблема аудитории), затем создаётся конкретное решение под неё
- **Итерационный подход** — цикл разработки: собрал MVP → получил обратную связь → доработал → показал снова
- **B2B (Business to Business)** — направление работы, при котором вы продаёте услуги или продукты бизнесу, а не частным лицам
- **Кейс** — оформленный пример выполненного проекта: проблема, решение, результат
- **Язык пользы** — способ описания своей работы через конкретный эффект для клиента, а не через технические детали

## Домашнее задание

Составь личный план развития на ближайшие 30 дней и начни его реализацию.

### Что нужно сделать:

1. Выбери один вектор развития из урока: фронтенд, агентные системы, автоматизация контента, SaaS, фриланс или карьера
2. Определи конкретный проект или задачу, которую ты будешь развивать в ближайший месяц
3. Заполни маршрут: для кого проект, какой сценарий должен работать, как ты проверишь результат, как будешь презентовать
4. Сделай первый шаг прямо сейчас — доработай финальный проект, улучши лендинг, добавь новый скилл в агента или запусти MVP для потенциального клиента

**Результат:** оформленный план на 30 дней + первое действие по нему уже выполнено.

В качестве домашнего задания прикрепи документ, в котором опиши план развития на 30 дней, укажи, как ты выполнил(а) первое действие по этому плану, и добавь подтверждающие скриншоты.

## Задание

Составь личный план развития на ближайшие 30 дней и начни его реализацию.

### Что нужно сделать:

1. Выбери один вектор развития из урока: фронтенд, агентные системы, автоматизация контента, SaaS, фриланс или карьера
2. Определи конкретный проект или задачу, которую ты будешь развивать в ближайший месяц
3. Заполни маршрут: для кого проект, какой сценарий должен работать, как ты проверишь результат, как будешь презентовать
4. Сделай первый шаг прямо сейчас — доработай финальный проект, улучши лендинг, добавь новый скилл в агента или запусти MVP для потенциального клиента

**Результат:** оформленный план на 30 дней + первое действие по нему уже выполнено.

В качестве домашнего задания прикрепи документ, в котором опиши план развития на 30 дней, укажи, как ты выполнил(а) первое действие по этому плану, и добавь подтверждающие скриншоты.